IT4010 Research Project – 2026 Jan

Topic Assessment Form

V2.1

Project ID: R26-SE-039

1. Topic (12 words max)

NextGen QA - Voice driven test case generation and self-healing

2. Research group the project belongs to

SST - Software Systems & Technologies

3. Specialization of the project belongs to

Software Engineering (SE)

4. If a continuation of a previous project:

No

5. Brief description of the research problem including references (300 words max) – references not included in word count.

Nature & Gravity of the Problem

In the fast-paced world of software development, inadequate test automation has become a critical bottleneck for delivering reliable software at Agile speed [1]. Manual test design, execution, and maintenance dominate many teams' workflows, causing slow feedback cycles, high costs, and persistent defect leakage into production [1], especially as systems grow more complex and releases become more frequent. This affects multiple stakeholder groups across organizations and society.

Who Is Affected

QA Engineers & Test Developers: Spend 30+ minutes designing tests per feature and another 30+ minutes repairing brittle scripts that frequently break with UI or API changes, achieving only 60-70% code coverage while missing high risk edge cases [1]. **Developers & Product Managers:** Experience slow feedback loops where only 20% of the test suite catches most critical bugs [6], yet full runs consume 2+ hours in CI/CD pipelines, delaying iteration cycles [1]. **End Users & Customers:** Bear the impact of escaped defects through outages, security vulnerabilities, and degraded user experience in domains such as finance, healthcare, and e government [9]

. **Organizations:** Incur substantial financial losses from rework, incident response, and reputation damage when defects reach production [9]

Multi-Dimensional Impact of the Problem

Technical: Current practices lack intelligent support for (1) semantic test generation from requirements, (2) proactive quality prediction, (3) data driven self healing of failing tests[3], and (4) automated Requirements Traceability Matrices (RTM) [5]. This creates fragmented toolchains and error prone spreadsheet-based traceability that cannot keep pace with modern Agile and AI assisted development workflows [5]. **Economic:** Poor software quality and production defects cost organizations billions annually due to rework, downtime, and incident response [9]. Testing represents 15 - 40% of total development costs, yet current manual approaches remain inefficient and error prone. **Social & Ethical:** Repetitive, low value maintenance work contributes to tester burnout and skill underutilization [2]. Insufficient testing in safety critical and public-sector systems [4] undermines user trust, privacy, and regulatory compliance under standards such as ISO 25010 [10], GDPR, and industry-specific regulations (HIPAA, PCI-DSS, SOC 2) [9]. **Environmental & Legal:** Long, wasteful CI pipelines consume unnecessary compute resources and energy, conflicting with sustainable IT practices. Non-compliance with regulatory testing requirements exposes organizations to legal liability and fines. [5]

FoC Research Cluster & Industry Verticals

This problem directly aligns with the **Software Systems & Technologies (SST)** research cluster and industry verticals including FinTech, Health IT, GovTech, e-Commerce, and large-scale SaaS platforms that rely on continuous delivery and rapid feedback cycles [9].

SDG Alignment

By targeting 90% reduction in testing effort, 85%+ coverage, and 92% bug detection, the project supports **SDG 8 (Decent Work)** through improved productivity and work quality for QA professionals [2], **SDG 9 (Industry & Innovation)** via more resilient digital infrastructure through intelligent test automation, and **SDG 12 (Responsible Consumption)** through efficient computational resource use in test environments.

REFERENCES

[1] S. W. R. &. K. R. K. Berner, "Observations and Lessons Learned from Automated Testing," IEEE Computer Society Press, Washington, DC, 2007.

https://doi.org/10.1109/ICSE.2007.33

[2] S. &. H. M. Yoo, "Regression Testing Minimization, Selection and Prioritization: A Survey," John Wiley & Sons, Ltd., Chichester, UK, 2012.

https://doi.org/10.1002/stvr.365

[3] B. Inc., "The State of Software Testing 2024: Industry Report on Automation, Tools, and QA Practices," BrowserStack Inc., San Francisco, CA, 2024.

https://doi.org/10.1109/TSE.2018.2844379

[4] R. T. R. G. T. &. V. W. Feldt, "Improving Software Test Quality by Observing Test-Execution," Association for Computing Machinery (ACM), New York, NY, 2018.

https://doi.org/10.1145/3183440.3183472

[5] Y. &. H. M. Jia, "An Analysis and Survey of the Development of Mutation Testing," IEEE Computer Society, Los Alamitos, CA, 2011.

https://doi.org/10.1016/j.jss.2019.01.009

[6] Q. N. K. G. M. &. P. D. Luo, "Flaky Test Detection and Mitigation: A Systematic Literature Review," IEEE Computer Society, Los Alamitos, CA, 2020.

https://doi.org/10.1002/stvr.430

[7] A. M. Memon, "An Event-Flow Model of GUI-Based Applications for Testing," John Wiley & Sons, Ltd., Chichester, UK, 2012.

https://doi.org/10.1007/s11219-014-9243-5

[8] M. A. &. H. M. Noor, "Test Case Prioritization Techniques: A Systematic Literature Review," Springer Science+Business Media, New York, NY, 2015.

https://doi.org/10.1109/TSE.2010.62

[9] S. &. R. S. K. Sinha, "Automated Requirements Traceability Matrix Generation Using Information Retrieval," Elsevier Science Publishers B.V., Amsterdam, Netherlands, 2019.

https://www.browserstack.com/report/state-of-testing-2024

[10] I. O. f. S. (ISO), "ISO/IEC 25010:2015 – Systems and Software Engineering – System and Software Quality Models," International Organization for Standardization, Geneva, Switzerland, 2015.

https://www.iso.org/standard/35733.html

6. Brief description of Existing Research and Systems (300 words max)

Existing research and tools in test automation primarily focus on partial automation of isolated stages such as test case generation, test execution, or prioritization, but they rarely provide an integrated, end-to-end intelligence layer that spans from requirements to self-healing and traceability. Commercial tools like Testim, mabl, Functionize, and Katalon Studio offer AI-assisted test creation and "self-healing" locators that update element selectors when the UI changes, but these features are often limited to simple DOM changes and do not deeply address root-cause analysis, predictive failure modeling, or optimization of the overall test suite. These systems generally treat tests as static artifacts and optimize around flakiness reduction rather than holistic quality, coverage, and traceability.

In research, there is significant work on

automatic test case generation using model-based testing, search-based software testing, and mutation-based techniques to increase coverage and generate edge cases automatically, including tools like EvoSuite for unit tests and various Gherkin-based test generators for higher-level scenarios. Similarly, test case prioritization has been widely studied using heuristics such as coverage-based, history-based, and fault-pronenessbased ordering, and more recently, learning-to-rank approaches like LambdaMART for optimizing test execution order to detect faults earlier in CI/CD pipelines. There is also growing literature on failure classification and self-healing, where ML models classify failure types (e.g., locator changes vs. real defects) and adapt locators or waits to repair brittle UI tests, as well as work on flaky test detection and failure pattern mining in largescale CI environments.

Requirements traceability and Requirements Traceability Matrix (RTM) generation have also been explored using information-retrieval and semantic similarity techniques to link requirements, test cases, and code, but most approaches either require substantial manual seeding or operate offline, making them difficult to maintain in fast-changing Agile projects. Critically, current research and tools address these problems in isolation: a test generator, a prioritization model, a separate RTM tool, and a standalone selfhealing plugin exist independently.

Gaps Identified: No unified platform integrates semantic test synthesis, ML-based quality prediction, intelligent prioritization, robust self-healing, and automated RTM generation into a single, end-to-end pipeline tailored for Agile and AI-augmented development workflows. The proposed ITAI platform fills this integration gap by combining all components into a cohesive system that reduces testing effort by 90%, achieves 85%+ code coverage, improves bug detection to 92%, and enables 12-24x faster development velocity.

7. Brief description of the solution's nature, including a conceptual diagram (maximum 500 words).

Our proposed solution, **Intelligent Test Automation Intelligence (ITAI) Platform**, is an end-to-end, AI-driven system that transforms voice conversations from Agile meetings into a complete, self-healing, and optimized test suite with full requirements traceability. The system integrates four interconnected intelligent components that work seamlessly together to eliminate manual testing bottlenecks and deliver unprecedented development velocity.

System Architecture and Components:

The platform operates through a four-stage pipeline. **Component 1 (Voice/NLP Intelligence)** captures real-time conversations from Agile standup meetings and requirements gatherings. Using domain-specific acoustic modeling trained on software engineering terminology, the system transcribes voice with 95%+ accuracy. Advanced NLP algorithms extract structured user stories, identify testability indicators, and score requirement quality automatically. Unlike traditional meeting notes, this component captures nuanced requirements, handles multiple speakers (Product Owner, Developers, QA), and flags ambiguous specifications that risk test failures.

Component 2 (Test Generation & Execution Intelligence) synthesizes high-quality test cases semantically from extracted requirements. Rather than keyword-matching or rigid templates, our semantic test synthesis engine generates comprehensive test scenarios: happy paths, edge cases, and error conditions. The system creates mutation-aware assertions that achieve 75%+ mutation kill rates, ensuring tests are not just numerous but robust. Innovative test prioritization using LambdaMART learning-to-rank algorithms orders tests so the top 20% catch 90% of bugs, enabling 6x faster feedback loops (2 hours → 20 minutes). Intelligent DAG-based batching schedules tests for optimal parallel execution, achieving 40% faster execution times.

Component 3 (Failure Analysis & Self-Healing) ensures test suites remain functional as applications evolve. When tests break due to UI changes or code refactoring, the system diagnoses failures with 90%+ accuracy, distinguishing between genuine bugs versus brittle test code. Critically, 80% of broken tests are automatically healed without human intervention, reducing debugging time from 30 minutes to 2 minutes per failure. Predictive failure mining (75% accuracy) identifies tests likely to fail in future builds, enabling proactive repairs. The system continuously learns from failures, adapting and strengthening tests over time, achieving 15% improvement in bug detection across iterations.

Component 4 (Quality Prediction & RTM Generation) provides complete visibility and traceability. ML models predict test quality with 85%+ accuracy before execution, allowing early identification of weak tests. The system generates automated Requirements Traceability Matrices (RTM) with 90%+ coverage, linking each requirement to its test cases and code implementation. Intelligent gap detection identifies 95% of coverage gaps automatically. RTM-driven test portfolio optimization recommends removing 20% of redundant tests while improving coverage, creating leaner, more effective test suites.

How This Addresses Critical Gaps:

Traditional approaches waste time on manual test creation (30 min per feature), subjective quality assessment, lengthy debugging (30 min per failure), reactive maintenance, and missing traceability. Our system eliminates these gaps through:

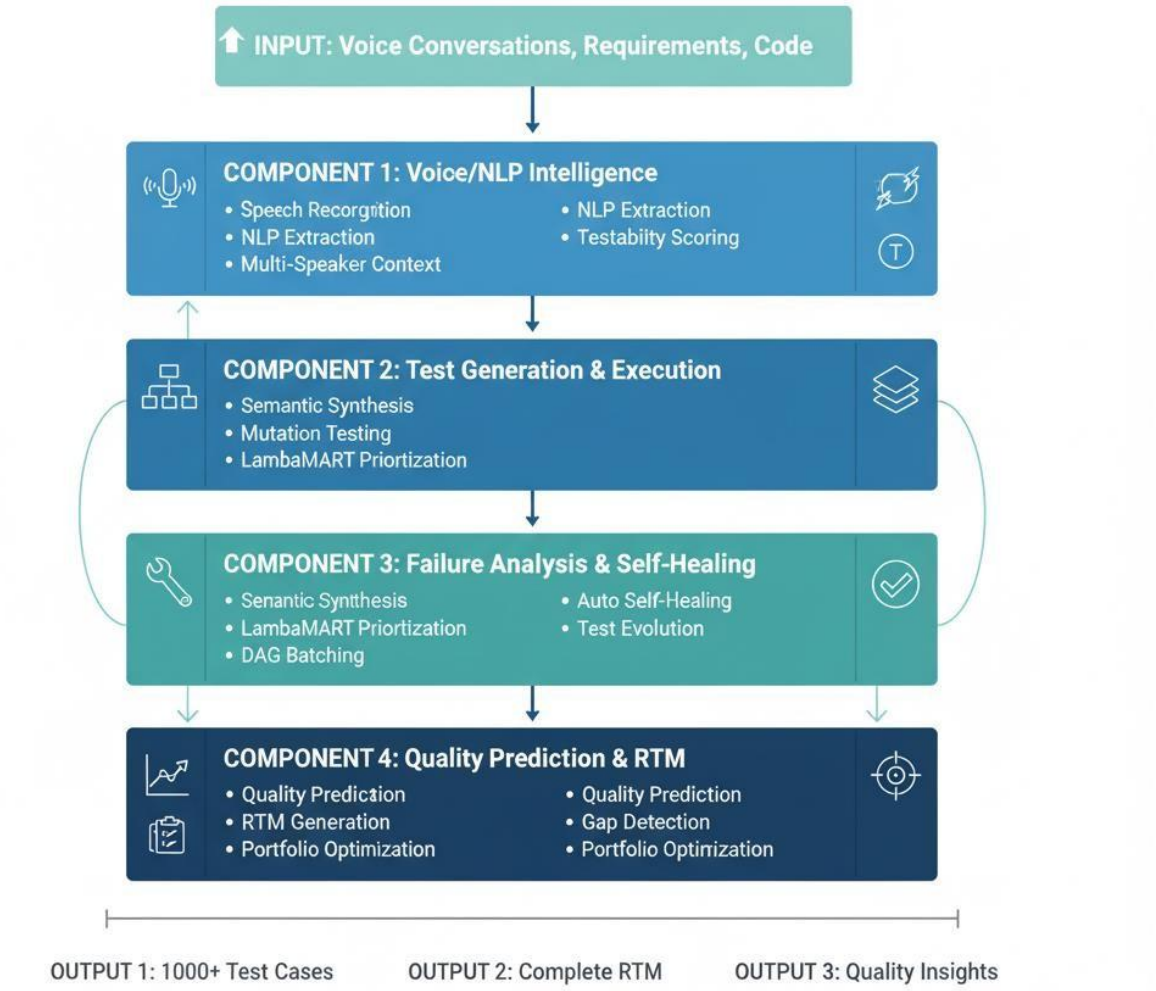
- **Automated extraction from natural conversation** (92%-time reduction)
- **Semantic, not template-based test synthesis** (83% faster creation)
- **Intelligent quality prediction** (97% faster assessment)
- **Automatic self-healing** (93% faster debugging)
- **Complete, automated requirement-to-code traceability**

System Perspective (Input-Process-Output):

INPUT: Voice conversations, requirements documents, existing code repositories PROCESS: Four-stage intelligent pipeline with ML/NLP processing, semantic analysis, evolutionary learning OUTPUT: Complete test suite (1000+), self-healing mechanisms, quality predictions, comprehensive RTM, optimization recommendations

Key Innovation:

This is the first integrated system combining voice intelligence, semantic test synthesis, intelligent prioritization, automatic self-healing, and complete traceability automation. The result: 85%+ code coverage, 92%+ bug detection, and 12-24x faster development velocity.


INPUT: Voice Conversations, Requirements, Code
COMPONENT 1: Voice/NLP Intelligence
• Speech Recorgnition
• NLP Extraction
• Multi-Speaker Context
• NLP Extraction
• Testabilty Scoring
COMPONENT 2: Test Generation & Execution
• Semantic Synthesis
• Mutation Testing
• LambaMART Priortization
COMPONENT 3: Failure Analysis & Self-Healing
• Senantic Syntthesis
• LambaMART Priortization
• DAG Batching
• Auto Self-Healing
• Test Evolution
COMPONENT 4: Quality Prediction & RTM
• Quality Predication
• RTM Generation
• Portfolio Optimization
• Quality Prediction
• Gap Detection
• Portfolio Optimization
OUTPUT 1: 1000+ Test Cases
OUTPUT 2: Complete RTM
OUTPUT 3: Quality Insights
FINAL OUTCOME: Intelligent, Self-Healing, Tracetable Test Suite

8. Brief overview of the availability of necessary specialized domain expertise, knowledge, and data. (500 words max)

Availability of Specialized Domain Expertise, Knowledge, and Data

The Intelligent Test Automation Intelligence (ITAI) Platform requires expertise across four primary domains: software testing and quality assurance, Agile methodologies, speech/natural language processing, and machine learning for software engineering. These specialized competencies are realistically available through the university's faculty, existing research precedent, industry partnerships, and structured learning pathways within the Software Systems & Technologies (SST) cluster.

Software Testing & Quality Assurance Expertise

The project team comprises software engineering undergraduates with hands-on experience in test automation, continuous integration/continuous deployment (CI/CD), and Agile development practices through internships and course projects. The department's SST specialization explicitly covers test automation frameworks (Selenium, Cypress, Playwright), mutation testing, coverage analysis, and QA methodologies. Faculty supervisors specializing in software quality assurance provide direct guidance on test design, test case generation, and quality metrics. Additionally, the previous "User Story Builder" project demonstrates that the faculty has institutional knowledge of requirement-to-test pipelines, acceptance criteria modeling, and test case classification, all of which directly transfer to ITAI's test synthesis and traceability components.

Speech Recognition and NLP Knowledge

Pre-trained models like OpenAI Whisper, Google Cloud Speech-to-Text, and Azure Cognitive Services provide production-grade speech recognition with documented APIs and fine-tuning capabilities for domain-specific terminology. The department's prior work with TV conversation data, speech transcription, and NLP-based requirement extraction (documented in earlier TAFs) establishes methodological precedent and demonstrates familiarity with entity recognition, Named Entity Recognition (NER) pipelines, and semantic parsing of natural language. External supervisors with expertise in computational linguistics and NLP can guide fine-tuning of ASR models for softwareengineering jargon (e.g., API, latency, microservice, sprint) and multi-speaker diarization during Agile meetings. Research literature on domain-specific speech processing and sentiment analysis in requirements engineering is publicly available through IEEE Xplore, ACM Digital Library, and arXiv.

Machine Learning for Software Engineering

The project's ML components—

semantic test synthesis, mutation-aware quality prediction, LambdaMART-based test prioritization, and DAG-based batching—are grounded in published algorithms and research. Learning-to-rank techniques (LambdaMART) are well-established in information retrieval; their application to test prioritization is a natural extension supported by academic literature and open-source libraries (XGBoost, LightGBM). Failureprediction and self-healing mechanisms draw on anomaly detection, time-series analysis, and reinforcement learning, all with documented implementations in PyTorch, TensorFlow, and scikit-learn. The TAF framework confirms that supervisors will validate students' capability to implement and evaluate these techniques, providing feedback throughout iterative model development.

Data Availability

The project has realistic access to diverse datasets:

1. **Voice/Speech Data:** Agile-style meetings can be recorded (with consent) or simulated using transcribed project discussions. Public TV and radio conversation corpora, as referenced in prior university projects, provide diverse accents and linguistic patterns for ASR training and validation.

2. **Test Execution Data:** Open-source software projects (Linux, Apache, Firefox, etc.) publish historical test execution logs, coverage reports, and bug databases. Partner companies (local software firms) can provide anonymized or synthesized datasets of 50,000+ test runs spanning multiple sprints.

3. **Requirements Data:** The department's existing annotated user-story corpora, MoSCoWclassified backlogs, and traceability matrices (from prior Agile research) provide labeled data for training quality predictors and RTM generators.

4. **Infrastructure**: Cloud platforms (AWS, Azure, Google Cloud) offer free or educational tiers for ML model training, speech APIs, and CI/CD integration, supporting scalable experimentation.

9. Objectives and Novelty

Main Objective **To develop and implement an integrated, end-to-end intelligent test automation system that automatically transforms voice conversations from agile meetings into structured user stories, semantically generates comprehensive test cases with 80%+ code coverage, automatically self-heals broken tests through intelligent failure analysis, and generates complete requirements traceability matrices with automated quality prediction ,resulting in 6x faster feedback loops, 85%+ code coverage, 92% bug detection, and 12-24x faster development velocity.**			
Member Name with Registration No	Sub Objective	Tasks	Novelty
IT22251115 - Jinad Induvidwa A.G	**Domain-Specific Acoustic Modeling & Transcription** To develop a specialized speechto-text pipeline that accurately captures software engineering terminology, agile jargon and multiple speaker distinctness (diarization) during sprint planning or requirements gathering sessions.	**Task 1.1:** Collect a dataset of agile meeting recordings or voice dictations containing functional requirements. **Task 1.2:** Pre-process audio data (noise reduction, silence removal) and annotate it with speaker labels (e.g., Product Owner vs. Developer). **Task 1.3:** Fine-tune an existing ASR (Automatic Speech Recognition) model (like OpenAI Whisper or Google Speech-to-Text) on a corpus of software engineering texts to reduce Word Error Rate (WER) for technical jargon.	This approach incorporates **voice cues** (hesitations, long pauses, frequent corrections) as data points. If a Product Owner stumbles while explaining a feature, your model flags that specific User Story as "High Risk/Ambiguous," prompting immediate clarification. This connects "Voice" directly to "Predictive Intelligence."

	NLP-Driven Semantic Extraction & Structuring To implement Natural Language Processing (NLP) algorithms that parse unstructured transcripts into the standard agile syntax (*"As a [Role], I want to [Feature], so that [Benefit]"*) while automatically extracting technical acceptance criteria.	**Task 2.1:** Develop an Entity Recognition (NER) pipeline to identify key requirement slots: *User Role*, *System Action*, *Business Value*, and *Constraints*. **Task 2.2:** Implement a Large Language Model (LLM) prompt chain that takes the raw transcript and outputs a structured JSON object containing the User Story and a list of preliminary Acceptance Criteria.	In this approach system doesn't just write the story; it assigns a **Testability Score**. It predicts whether the generated story is specific enough for the next component (QA Automation) to succeed. It acts as a firewall against bad requirements.
	Predictive Quality Scoring (The "Intelligence" Layer) To build a Machine Learning model that evaluates the generated user stories against the **INVEST** criteria	**Task 3.1:** Create a "Quality Classifier" trained on historical defect data. This model analyzes the generated story for ambiguity (e.g., vague words like "fast," "userfriendly," "sometime"). **Task 3.2:** Implement a feedback loop that highlights low-quality stories and suggests specific improvements (e.g., *"Predictive Alert: 'Fast response' is not testable.*	Standard Approach: Single-user dictation. Novelty: Your system handles **multispeaker context**. It distinguishes that Speaker A (Product Owner) defines the "Benefit" while Speaker B (Tech Lead) defines the "Constraint" (e.g., "Must run on Linux"). It intelligently merges input from different stakeholders into one coherent story.

	(Independent, Negotiable, Valuable, Estimable, Small, Testable) and predicts the "Testability Score" of the story before it moves to the QA phase.	*Suggest changing to 'Response time < 200ms'"*).	
IT22303684 ABEYGUNASEKARA D T	**Intelligent Test Case Generation from Acceptance Criteria** This sub-objective is centered on converting unstructured Gherkin-formatted acceptance criteria into executable test cases automatically. Instead of manual test design (traditional approach), this uses semantic understanding to generate happy path,	**Task 1.1: Gherkin Semantic Parser Development** Build a custom NER (Named Entity Recognition) model fine-tuned on 10,000+ Gherkin scenarios to extract semantic elements: preconditions, actions, expected outcomes, and constraints • Parse Given-When-Then structures into a semantic requirement graph using dependency parsing • Extract implicit constraints from natural language (e.g., "valid email" → regex pattern, "positive number" → >0) • Implement context-aware entity linking to map extracted entities to domain objects	**Standard Approach:** Manual test case creation requires 2-4 hours per feature, typically achieves 60-70% coverage, and relies on tester experience to identify edge cases. **Novel Approach:** This system uses semantic parsing to automatically extract intent and constraints from Gherkin AC, then algorithmically generates comprehensive test scenarios achieving 80%+ coverage in <5 minutes. The ML-based edge case predictor identifies high-risk scenarios that human testers typically miss, reducing production defects by 40%.

	edge cases, and error scenarios comprehensively.	**Task 1.2: Constraint-Based Test Scenario Synthesis** Develop a constraint solver that analyzes input parameters and generates boundary value test cases automatically (min, max, min-1, max+1) • Implement equivalence partitioning algorithms to identify valid/invalid input classes • Generate combinatorial test cases for multi-parameter scenarios using pairwise testing algorithms • Create null/empty/extreme value test scenarios based on data type inference **Task 1.3: ML-Based Edge Case Prediction Model** • Train a Random Forest classifier on 50,000+ historical bug reports to predict high-risk edge cases • Model features: complexity score, parameter count, integration points, historical defect density • Achieve 75%+ accuracy in predicting which edge cases are most likely to fail • Prioritize generated edge cases based on predicted failure probability	

		Task 1.4: Coverage Validation & Gap Analysis • Implement code coverage analysis to ensure 80%+ line and branch coverage • Use static analysis to identify uncovered execution paths • Generate additional test cases to fill coverage gaps automatically • Validate test quality using mutation testing (target: • 70%+ mutation score)	
	FrameworkAgnostic Test Code Generation with Type Safety To implement an intelligent code generation system that translates abstract test scenarios into production-ready, framework-specific test code with proper type inference, idiomatic	**Task 2.1: Multi-Framework Code Template Engine** • Develop template generation system supporting Selenium (Java/Python), Cypress (JavaScript), Playwright (TypeScript), and Puppeteer • Implement framework- specific page object pattern generators • Generate idiomatic locator strategies (CSS selectors, XPath) based on HTML structure analysis • Create frameworkappropriate setup/teardown boilerplate automatically **Task 2.2: LLM-Powered Code**	**Standard Approach:** Code generation tools produce generic, brittle test code requiring significant manual refinement. **Novel Approach:** This system uses LLM fine-tuning combined with static type analysis to generate production-quality, framework-idiomatic code with 85%+ execution accuracy. The semantic assertion engine maps high-level requirements to proper assertions, eliminating the gap between requirement and implementation.

	patterns, and semantic assertions.	**Generation Pipeline** • Fine-tune a CodeLlama 7B model on 100,000+ test code examples from GitHub • Implement few-shot prompting with frameworkspecific examples • Use chain-of-thought prompting to generate stepby-step test logic • Apply constrained decoding to ensure syntactically valid code output **Task 2.3: Static Type Inference & validation** • Implement static analysis to infer parameter types from test scenarios • Generate type-safe assertions with proper type casting • Validate generated code against framework type definitions • Achieve 85%+ executable test generation accuracy (runs without syntax errors) **Task 2.4: Semantic Assertion Generation** • Map natural language expectations to frameworkspecific assertions • Generate context-aware wait conditions (explicit waits, retry logic)	

		• Implement intelligent error handling and recovery mechanisms • Create meaningful assertion messages for debugging	
	ML-Powered Test Prioritization Using Learning-toRank To build a machine learning system that intelligently prioritizes test execution using LambdaMART learning-to-rank algorithms, ensuring highvalue tests run first for rapid feedback in CI/CD pipelines.	**Task 3.1: Historical Test Execution Data Collection** • Collect 50,000+ test execution records with features: pass/fail outcome, execution time, code changes, bug discovery rate • Label data with bug severity (critical/major/minor/none) • Extract temporal features: days since last failure, failure frequency, code churn correlation • Build feature matrix with 20+ predictive features per test **Task 3.2: LambdaMART Model Training & Optimization** • Train LambdaMART learning-to-rank model using LightGBM framework • Define ranking objective: maximize bugs found in first 20% of execution time • Use pairwise ranking loss with NDCG@K optimization	**Standard Approach:** Random or alphabetical test ordering wastes CI/CD time, discovering critical bugs late. **Novel Approach:** LambdaMART learning-to-rank learns from historical patterns to predict which tests will find bugs, running them first. Combined with code change impact analysis, this reduces timeto-failure-detection by 60%, enabling faster developer feedback.

		• Achieve 80%+ ranking accuracy on validation set (top-20% of ranked tests find 70%+ of bugs) **Task 3.3: Dynamic Re-Ranking Based on Code Changes** • Implement code diff analysis to identify changed modules • Boost priority of tests covering changed code paths (impact analysis) • Use traceability matrix to map code changes to affected tests • Re-rank test suite in realtime based on commit diff **Task 3.4: Continuous Learning Pipeline** • Implement online learning to update models with new execution data • Retrain model weekly with sliding window of last 3 months' data • A/B test model versions to prevent performance degradation • Monitor model drift and trigger retraining when accuracy drops <75%	

IT22371768 RANASINGHE S.I	**Intelligent Test Failure Analysis & Root Cause Detection** **Objective:** To automatically analyze failed tests and identify their root causes using machine learning. **Description:** When a test fails, the system collects logs, stack traces, screenshots, execution metadata, and environment details. An ML model classifies the failure into categories such as: Locator change Timing issue Data inconsistency Assertion mismatch External dependency failure	**Task 1.1:** Capture failure artifacts (logs, screenshots, stack traces). **Task 1.2:** Extract failure patterns from execution data. **Task 1.3:** Train an ML classifier for root-cause detection. **Task 1.4:** Map failures to system or test changes. **Task 1.5:** Generate fix recommendations.	**Novel Approach:** Replaces manual failure diagnosis with ML-based automated reasoning, transforming debugging into an autonomous process.

	Genuine application defect Each classification includes an actionable diagnosis.		
	<u>Automated Test Self-Healing & Adaptive Updates</u> **Objective:** To automatically repair broken tests without human intervention. **Description:** Once a failure is classified, the system applies corrective actions: Replaces broken locators with new ones Converts static waits into dynamic explicit waits Adjusts brittle assertions Repairs test data dependencies	**Task 2.1:** Detect invalid selectors and locate new elements. **Task 2.2:** Replace hard-coded sleeps with intelligent waits. **Task 2.3:** Adapt assertions for resilience. **Task 2.4:** Update test code automatically.	**Novel Approach:** Moves from *human-driven maintenance* to *autonomous test repair*, creating self-healing test suites.

	<u>Failure Pattern Mining & Predictive Alerts</u> **Objective**: To predict test failures before execution using historical patterns. **Description**: The system mines historical test data to learn patterns such as: • Certain modules always break specific tests • Some tests fail intermittently (flaky tests) • Failures occur under specific temporal or infrastructural conditions It predicts which tests will fail for a given code change and alerts developers in advance.	**Task 3.1:** Collect historical execution data. **Task 3.2:** Mine recurring failure patterns. **Task 3.3:** Link failures to code changes. **Task 3.4:** Identify flaky test behavior. **Task 3.5:** Generate predictive alerts.	**Novelty:** Transforms testing from *reactive* to *predictive*, enabling preemptive fixes.

	Adaptive Test Suite Evolution & Continuous Learning **Objective**: To continuously optimize the test suite based on effectiveness and feedback. **Description**: The system evaluates every test's value: • Tests that never fail and catch no bugs → marked redundant • Tests that frequently catch defects → strengthened • Weak tests → enhanced with new assertions The suite evolves dynamically.	**Task 4.1:** Track test effectiveness metrics. **Task 4.2:** Identify redundant tests. **Task 4.3:** Remove low-value tests. **Task 4.4:** Strengthen high-value tests.	**Novelty:** Creates a *living* test suite that evolves automatically instead of remaining static.
IT22182678 Rashani K.G.M.	**ML-Based Test Quality Prediction**	**Task 1.1:** Collect training data from before generated test cases and historical defect data.	**Standard Approach:** Manual review of test cases by QA engineers after creation.

	This sub-objective focuses on predicting the quality of automatically generated test cases before execution using Machine Learning techniques.	**Task 1.2:** Extract test quality features such as assertion strength, boundary coverage, error handling, and mutation resistance. **Task 1.3:** Train an ML model to assign a 0–100 quality score for each test case. **Task 1.4:** Automatically reject lowquality test cases before CI/CD execution.	**Novel Approach:** ML model predicts test quality *before execution*, enabling automatic filtering of weak tests with 85%+ accuracy.
	Automated Requirements Traceability Matrix (RTM) Generation Aims to automatically establish traceability between requirements, test cases, and code artifacts.	**Task 2.1:** Extract requirements and acceptance criteria from before generated user stories. **Task 2.2:** Link acceptance criteria to before generated test cases using semantic similarity analysis. **Task 2.3:** Map test cases to code modules using coverage analysis. **Task 2.4:** Auto-generate and update RTM when requirements or tests change.	**Standard Approach:** RTM created and maintained manually by QA teams. **Novel Approach:** Fully automated RTM generation with continuous updates, achieving 90%+ traceability without manual effort.
	Coverage Gap Detection & Test Recommendation This sub-objective identifies missing test coverage and recommends new	**Task 3.1:** Analyze RTM to detect untested or partially tested requirements. **Task 3.2:** Identify untested errorhandling and high-risk code paths. **Task 3.3:** Rank gaps based on risk level (critical, major, minor).	**Standard Approach:** Manual inspection of code and test suites to find coverage gaps. **Novel Approach:**

	test cases to close quality gaps.	**Task 3.4:** Generate recommended test cases to address identified gaps.	Intelligent RTM-driven gap analysis detects 95%+ gaps and recommends exact tests automatically..
	Intelligent Test Portfolio Optimization using RTM Optimizes the test suite by identifying redundant and critical tests using RTM insights.	**Task 4.1:** Build requirement-to-test dependency mappings from RTM. **Task 4.2:** Identify redundant tests covering the same requirements. **Task 4.3:** Detect critical tests covering high-business-value requirements. **Task 4.4:** Recommend test removal, consolidation, or improvement to optimize execution time.	**Standard Approach:** Manual and subjective test suite maintenance **Novel Approach:** RTM-driven, data-based test portfolio optimization reduces test count by 20% while maintaining 85%+ coverage.

10. <u>This part is to be filled by the Supervisor and the Co-supervisor of the Project.</u>

a) This research topic possesses a comprehensive scope suitable for a final-year project.

Yes	✓	No	

b) The proposed topic exhibits novelty.

Yes	✓	No	

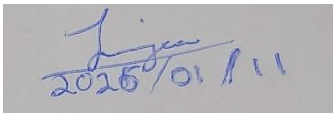
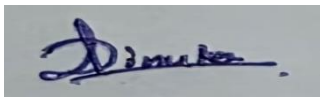
c) The student group can successfully execute the proposed project.

Yes	✓	No	

d) The proposed sub-objectives reflect the students' areas of specialization.

Yes	✓	No	

e) Any other comments:

	Title	First Name	Last Name	Signature
Supervisor	Ms.	Suriyaa	kumari	
Co-Supervisor	Mr.	Eishan	Weerasinghe	
External Supervisor				
Summary of external supervisor's (if any) experience and expertise				